



Security Assessment



Polygon Vault Bridge V1.1

November 2025

Prepared for Polygon

Table of content

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	4
Findings Summary.....	6
Severity Matrix.....	6
Detailed Findings.....	7
Medium Severity Issues.....	9
M-01. Migration may complete for wrong vbToken if mapping changes between asset and message claims.....	9
M-02. setCustomToken() during in-progress migrations can strand removal and block completion.....	10
M-03. Migration in-progress accounting couples assets and shares.....	11
M-04. Non-revoked yieldVault approval during pause undermines incident response.....	13
Low Severity Issues.....	14
L-01. Withdraw may deliver fewer assets than requested and event may misreport.....	14
L-02. Yield recipient cannot be updated when paused while yield collection remains callable.....	15
L-03. Migration completion on Primary Chain blocked by pause.....	16
L-04. Agglayer mint-to-zero path for migration finalization blocked by pause.....	17
L-05. Misleading decimals assertion may mask misconfiguration.....	18
Informational Issues.....	19
I-01. Polygon provider leaves setNativeConverter() enabled though unused.....	19
I-02. Minimum deposit threshold applied during rebalancing.....	19
I-03. Missing rotation runbook for LayerZero adapter handover.....	20
I-04. Use of ReentrancyGuardTransientUpgradeable breaks cross-chain compatibility.....	20
I-05. Event hygiene for admin/config setters (missing or insufficient emits).....	21
I-06. Non-standard ERC-20 Transfer(address(0), address(0), value) emitted.....	22
I-07. Zero-address mint path triggering migration finalization is permitted to NativeConverter.....	22
Disclaimer.....	23
About Certora.....	23

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
Polygon Vault Bridge V1.1	https://github.com/agglayer/vault-bridge/tree/feat/layerzero https://github.com/agglayer/vault-bridge/tree/tmp-lz-conflicts	7289ca3 (original commit) 950e64d (final commit)	Solidity

Project Overview

This document describes the manual code review findings of **Polygon Vault Bridge V1.1**. The review involved a re-audit of the full scope with a focus on the LayerZero integration. The following contract list is included in our scope:

```
src/etc/ERC20PermitUser.sol
src/etc/IAgglayerBridge.sol
src/etc/IBridgeMessageReceiver.sol
src/etc/IFiatTokenV2_2.sol
src/etc/IWETH9.sol
src/etc/IVaultBridgeTokenInitializer.sol
src/etc/InitializationCounterUpgradeable.sol
src/etc/Versioned.sol
src/primary-chain/MigrationManager.sol
src/primary-chain/VaultBridgeToken.sol
src/primary-chain/VaultBridgeTokenInitializer.sol
src/primary-chain/VaultBridgeTokenPart2.sol
src/primary-chain/ethereum/GenericVaultBridgeToken.sol
src/primary-chain/ethereum/vbETH/VbETH.sol
src/primary-chain/layerzero/NonDefaultOftAdapter.sol
```



```
src/secondary-chain/CustomToken.sol
src/secondary-chain/CustomTokenWethExtension.sol
src/secondary-chain/NativeConverter.sol
src/secondary-chain/agglayer/CustomTokenAgglayer.sol
src/secondary-chain/agglayer/GenericCustomTokenAgglayer.sol
src/secondary-chain/agglayer/GenericNativeConverterAgglayer.sol
src/secondary-chain/agglayer/NativeConverterAgglayer.sol
src/secondary-chain/agglayer/vbETH/WethAgglayer.sol
src/secondary-chain/agglayer/vbETH/WethNativeConverterAgglayer.sol
src/secondary-chain/agglayer/vbUSDC/bridged-usdc-standard/VbUsdcNativeConv
erterAgglayerBridgedUsdcStandard.sol
src/secondary-chain/layerzero/CustomTokenLayerZero.sol
src/secondary-chain/layerzero/GenericCustomTokenLayerZero.sol
src/secondary-chain/layerzero/NonDefaultMintBurnOftAdapter.sol
src/secondary-chain/layerzero/vbETH/WethLayerZero.sol
src/secondary-chain/polygon/CustomTokenPolygon.sol
src/secondary-chain/polygon/GenericCustomTokenPolygon.sol
src/secondary-chain/wormhole/CustomTokenWormhole.sol
src/secondary-chain/wormhole/GenericCustomTokenWormhole.sol
src/secondary-chain/wormhole/vbETH/WethWormhole.sol
```

The work was undertaken from **November 03, 2025**, to **November 17, 2025**. During this time, Certora's security researchers performed a manual audit of all the Solidity contracts and discovered several bugs in the codebase, which are summarized in the subsequent section.

Protocol Overview

Vault Bridge enables chains and apps to earn native yield on bridged TVL by staking underlying assets on the Primary Chain while keeping liquid representations on Secondary Chains.

The protocol is comprised of:

- One Primary Chain
 - Vault Bridge Token (vbToken): ERC-20/4626 vault that locks underlyings, optionally bridges minted vbToken, and manages yield.

- Migration Manager: singleton that receives cross-chain messages and finalizes migrations from Native Converters via a strict origin allowlist.
- Many Secondary Chains
 - Custom Token: chain-local representation minted/burned by the bridge or its adapter.
 - Native Converter (optional): converts between Custom Token and its underlying on the Secondary Chain, tracks backing, and can migrate backing to Primary.

Select assets are deposited into the Vault Bridge Token on the Primary Chain, which mints vbToken and can bridge it to Secondary Chains through supported providers (e.g., Agglayer/Polygon zkEVM, LayerZero, Wormhole) via adapters. Underlyings are put to work in a configured ERC-4626 yield vault on Primary, while vbTokens/Custom Tokens circulate in DeFi on Secondary. Produced yield can be collected on Primary and routed to a designated recipient.

A Native Converter on a Secondary Chain enables local acquisition/redemption of Custom Token without first bridging from Primary. As backing accumulates, the converter allows its migration to Vault Bridge Token on Primary via Migration Manager.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	4	4	2
Low	5	5	3
Informational	7	7	5
Total	16	16	10

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Migration may complete for wrong vbToken if mapping changes between asset and message claims	Medium	Acknowledged
M-02	<code>setCustomToken()</code> during in-progress migrations can strand removal and block completion	Medium	Fixed
M-03	Migration in-progress accounting couples assets and shares	Medium	Acknowledged
M-04	Non-revoked <code>yieldVault</code> approval during pause undermines incident response	Medium	Fixed
L-01	Withdraw may deliver fewer assets than requested and event may misreport	Low	Acknowledged
L-02	Yield recipient cannot be updated when paused while yield collection remains callable	Low	Fixed
L-03	Migration completion on Primary Chain blocked by pause	Low	Acknowledged
L-04	Agglayer mint-to-zero path for migration finalization blocked by pause	Low	Fixed

L-05	Misleading decimals assertion may mask misconfiguration	Low	Fixed
I-01	Polygon provider leaves <code>setNativeConverter()</code> enabled though unused	Informational	Fixed
I-02	Minimum deposit threshold applied during rebalancing	Informational	Fixed
I-03	Missing rotation runbook for LayerZero adapter handover	Informational	Acknowledged
I-04	Use of <code>ReentrancyGuardTransientUpgradeable</code> breaks cross-chain compatibility	Informational	Fixed
I-05	Event hygiene for admin/config setters (missing or insufficient emits)	Informational	Acknowledged
I-06	Non-standard ERC-20 <code>Transfer(address(0), address(0), value)</code> emitted	Informational	Fixed
I-07	Zero-address mint path triggering migration finalization is permitted to <code>NativeConverter</code>	Informational	Fixed

Medium Severity Issues

M-01. Migration may complete for wrong vbToken if mapping changes between asset and message claims

Severity: **Medium**

Impact: **High**

Likelihood: **Low**

Files: [MigrationManager.sol](#)
[VaultBridgeToken.sol](#)

Status: Acknowledged

Description: On Primary Chain, `MigrationManager.onMessageReceived()` selects the vbToken using the current `(originNetwork, originAddress) → TokenPair` mapping and calls `completeMigration()`. If a call to `configureNativeConverters()` happens between an asset and message claim, completion will use the new vbToken. In that scenario, if the new vbToken uses the same underlying, `completeMigration()` succeeds and vbToken is minted/bridged for the wrong token, misattributing assets. If the underlying differs, it reverts, leaving assets in MigrationManager. This involves distinct roles (Primary DEFAULT_ADMIN_ROLE vs MIGRATOR_ROLE) and can arise solely from insufficient cross-role coordination.

Recommendations: Encode the expected vbToken address into the migration message and require equality in `onMessageReceived()` before calling `completeMigration()`. Alternatively, document and enforce an operational runbook, such as:

- Revoke `MIGRATOR_ROLE` on the affected `NativeConverter` contracts.
- Finish any pending migrations for the pair.
- Pause `MigrationManager`.
- Update the mapping, then unpause `MigrationManager` and restore `MIGRATOR_ROLE`.

Customer's response: A complete revamp of Native Converter / backing migration mechanisms is planned for V2. The current mechanism will be replaced with a different one. In the meantime, we will keep this in mind.

Fix Review: Acknowledged.

M-02. `setCustomToken()` during in-progress migrations can strand removal and block completionSeverity: **Medium**Impact: **High**Likelihood: **Low**Files: [NativeConverter.sol](#)
[CustomTokenAgglayer.sol](#)

Status: Fixed

Description: `NativeConverter.setCustomToken()` checks `backingOnSecondaryChain == 0` (and gas backing) but not in-flight migrations. If `customToken` is swapped while `_migrationsInProgressCount > 0` or before pending zero-address mints arrive, the subsequent zero-address mint on the old token calls `removeMigrationInProgress()` via the old token, but the converter now enforces `onlyCustomToken` for the new token and reverts `Unauthorized`. This strands `_migrationsInProgress*` and blocks clean completion. This involves distinct roles (Secondary `DEFAULT_ADMIN_ROLE` vs `MIGRATOR_ROLE`) and can arise solely from insufficient coordination.

Recommendations: Prevent `setCustomToken()` when migrations are in-flight by requiring `_migrationsInProgressCount == 0`. Alternatively, document and enforce an operational runbook, such as:

- Freeze new starts: revoke `MIGRATOR_ROLE`.
- Require no in-flight: `_migrationsInProgressCount == 0` and `_totalMigratedBackingInProgress == 0`.
- Ensure no pending zero-address mints for that token/network on the bridge.
- Then call `setCustomToken` and re-grant roles.

Customer's response: Addressed in [c452808](#).

Fix Review: Confirmed.

M-03. Migration in-progress accounting couples assets and shares

Severity: **Medium**

Impact: **High**

Likelihood: **Low**

Files: [NativeConverter.sol](#)
[CustomTokenAgglayer.sol](#)

Status: Acknowledged

Description: The migration “in-progress” tracking in `NativeConverter` increments using migrated backing (`assets`) while the removal path decrements using minted custom token (`shares`). This implicitly assumes a permanent 1:1 exchange rate between assets and shares. If the ratio or decimals ever diverge, the map key will not match and totals may underflow or desync, blocking migration completion or corrupting accounting.

JavaScript

```
function _addMigrationInProgress(uint256 migratedBacking) internal {
    NativeConverterStorage storage $ = _getNativeConverterStorage();

    $_migrationsInProgress[migratedBacking]++;
    $_migrationsInProgressCount++;
    $_totalMigratedBackingInProgress += migratedBacking;

    emit MigrationInProgressAdded(migratedBacking);
}

function _removeMigrationInProgress(uint256 mintedCustomToken) private {
    NativeConverterStorage storage $ = _getNativeConverterStorage();

    $_migrationsInProgress[mintedCustomToken]--;
    $_migrationsInProgressCount--;
    $_totalMigratedBackingInProgress -= mintedCustomToken;

    emit MigrationInProgressRemoved(mintedCustomToken);
}
```

```
// in CustomTokenAgglayer.mint(...)
if (account == address(0)) {
    NativeConverter(nativeConverter()).removeMigrationInProgress(value);
    emit Transfer(address(0), address(0), value);
    return;
}
```

Recommendations: Convert shares to assets before subtracting, and always index and update the in-progress map and totals in assets so add/remove use the same unit. Include a lightweight check that shares and assets remain 1:1 and if the invariant ever changes, migrate to recording both units.

Customer's response: The units are different because backing is migrated (assets) and vbTokens are received back (shares). Therefore, the exchange between NC and VB is an 1:1 assets for shares exchange.

Fix Review: Acknowledged.

M-04. Non-revoked `yieldVault` approval during pause undermines incident responseSeverity: **Medium**Impact: **High**Likelihood: **Low**Files: [VaultBridgeTokenInitializer.sol](#)
[VaultBridgeTokenPart2.sol](#)

Status: Fixed

Description: `vbToken` grants the `yieldVault` an unlimited allowance of the underlying at initialization and does not revoke it on `pause()`. If the `yieldVault` is compromised, an admin response that pauses and then calls `drainYieldVault()` (both of which can be immediately executed, as opposed to `setYieldVault()` which requires configuration) will concentrate assets on `vbToken`. This creates a window where the compromised `yieldVault` can immediately `transferFrom` the drained underlying before the admin manages to call `setYieldVault()`, where approval is revoked.

Recommendation: Call `forceApprove(yieldVault, 0)` on `pause()` and re-grant the allowance on `unpause()`.

Customer's response: Auxiliary functions `revokeYieldVaultApproval()` and `restoreYieldVaultApproval()` have been implemented in [e0fa943](#), which permit revoking the approval without delay.

Fix Review: Confirmed.

Low Severity Issues

L-01. Withdraw may deliver fewer assets than requested and event may misreport

Severity: **Low**

Impact: **Medium**

Likelihood: **Low**

Files: [VaultBridgeToken.sol](#)

Status: Acknowledged

Description: In `VaultBridgeToken`, the withdraw path assumes the yield vault transfers exactly the requested amount of assets. The contract (a) forwards whatever the vault actually returned to the user (via `receivedAssets` plus any reserve) without enforcing equality to the requested `assets`, and (b) emits the ERC-4626 Withdraw event with the requested `assets` value rather than the measured delivery. The solvency check in `_withdrawFromYieldVault` constrains exchange-rate/solvency using requested `assets` and burned shares, not the actual delivered amount; therefore, it does not prevent under-delivery. Under a trusted ERC-4626, this is not expected to occur. However, if a non-compliant vault underdelivers, the user can receive fewer assets than requested while the event reports the full requested amount.

Recommendations: Consider asserting that delivered assets (`amountToWithdraw + receivedAssets`) equal requested `assets`, and emit the actual delivered assets.

Customer's response: only vaults that are ERC-4626 compliant and always send exactly `assets` will be used as `yieldVault`.

Fix Review: Acknowledged.

L-02. Yield recipient cannot be updated when paused while yield collection remains callableSeverity: **Low**Impact: **Low**Likelihood: **Low**Files: [VaultBridgeTokenPart2.sol](#)

Status: Fixed

Description: The `collectYield` function in `VaultBridgeTokenPart2` is intentionally callable during pause, allowing the system to continue transferring yield. However, the admin-only `setYieldRecipient` is `whenNotPaused`, preventing operators from switching the recipient during an incident without opening an unpause window. This can complicate incident response and routing updates when pausing is required for safety.

Recommendations: Allow changing the yield recipient during pause.

Customer's response: Addressed in [a603746](#) and [fcc991](#).

Fix Review: Confirmed.

L-03. Migration completion on Primary Chain blocked by pause

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files: [MigrationManager.sol](#)
[VaultBridgeTokenPart2.sol](#)

Status: Acknowledged

Description: On Primary Chain, completion of migrations is blocked while the system is paused because both the Agglayer Bridge callback `MigrationManager.onMessageReceived()` and the completion routine `VaultBridgeTokenPart2.completeMigration()` are guarded by `whenNotPaused`. In contrast, on Secondary Chains `NativeConverter.migrateBackingToPrimaryChain()` is callable while paused, so migrations can be initiated but not finalized during a pause. This creates an emergency-time liveness gap: underlying assets can accumulate at `MigrationManager` and cross-chain reconciliation cannot be executed until unpaused. This also prevents safely calling `configureNativeConverters()` as per M-01.

Recommendations: Permit migration completion while paused by removing `whenNotPaused` from `MigrationManager.onMessageReceived()` and `VaultBridgeTokenPart2.completeMigration()` so user flows remain paused while recovery stays live.

Customer's response: This is by design. Completing a migration is equivalent to depositing assets, which is disallowed when the protocol is paused.

Fix Review: Acknowledged.

L-04. Agglayer mint-to-zero path for migration finalization blocked by pauseSeverity: **Low**Impact: **Low**Likelihood: **Low**Files: [CustomTokenAgglayer.sol](#)
[NativeConverter.sol](#)

Status: Fixed

Description: On Agglayer, `CustomTokenAgglayer.mint()` is `whenNotPaused`. The migration-completion path relies on the “mint-to-zero” special case to invoke `NativeConverter.removeMigrationInProgress()`. When the token is paused, the mint-to-zero branch cannot execute, leaving `_migrationsInProgress` counters uncleared. This creates stale “migration in progress” state and operational ambiguity during incident response.

Recommendations: Allow mint-to-zero to bypass pause by handling the `account == address(0)` branch before the `whenNotPaused` check. This lets `removeMigrationInProgress(value)` run even when paused, while all other mint/burn paths remain pause-gated.

Customer’s response: Addressed in [7602e91](#).

Fix Review: Confirmed.

L-05. Misleading decimals assertion may mask misconfiguration

Severity: **Low**

Impact: **Low**

Likelihood: **Low**

Files: [GenericCustomTokenAgglayer.sol](#)
[WethAgglayer.sol](#)

Status: Fixed

Description: Both `reinitialize2()` implementations assert that `originalUnderlyingTokenDecimals_` equals `ERC20Upgradeable.decimals()`. However, `ERC20Upgradeable.decimals()` is a base implementation that always returns 18 and does not reflect the actual token metadata or preserved bridged token state. This assertion can provide false assurance, wrongly constraining non-18-decimals tokens or silently passing when metadata is not truly validated at runtime.

JavaScript

```
function reinitialize2(...) ... {  
    ...  
    string memory name_ = ERC20Upgradeable.name();  
    string memory symbol_ = ERC20Upgradeable.symbol();  
    assert(ERC20Upgradeable.decimals() == originalUnderlyingTokenDecimals_);  
    __CustomToken_init1(owner_, name_, symbol_, originalUnderlyingTokenDecimals_, bridge_,  
    nativeConverter_);  
}
```

Recommendations: Compare against `IERC20Metadata(address(this)).decimals()` rather than the base implementation.

Customer's response: Addressed in [f5a6177](#) and [44c2cf1](#). The old storage lives in a different slot.

Fix Review: Confirmed.

Informational Issues

I-01. Polygon provider leaves `setNativeConverter()` enabled though unused

Description: In the Polygon PoS child token variant, minting is restricted to the `ChildChainManager` and burning is self-initiated. The base token still exposes a one-time `setNativeConverter()`, which is not used by this provider model.

Recommendations: For parity and clarity, override `setNativeConverter()` in `CustomTokenPolygon` to revert, as is done in other providers.

Customer's response: Addressed in [00b3208](#).

Fix Review: Confirmed.

I-02. Minimum deposit threshold applied during rebalancing

Description: The documentation for `minimumYieldVaultDeposit` in `VaultBridgeToken` states "The limit does not apply when rebalancing the reserve." However, the rebalancing-down path calls `_depositIntoYieldVault(excess, false)`, which enforces the `minimumYieldVaultDeposit` threshold. As a result, when the reserve is slightly above target and `excess < minimumYieldVaultDeposit`, the deposit is skipped and the small surplus remains in `reservedAssets`. This is operationally benign, but it contradicts the stated behavior and can leave small amounts idle instead of being deployed.

Recommendations: Align behavior and documentation: either (a) explicitly bypass the threshold during `_rebalanceReserve()` when depositing `excess`, or (b) update the docs to state that the threshold applies during rebalancing as well.

Customer's response: Addressed in [1ec5b2e](#).

Fix Review: Confirmed.

I-03. Missing rotation runbook for LayerZero adapter handover

Description: The LayerZero README references wiring changes via the LayerZero CLI, and the USDC README outlines a high-level handover (“double totalSupply and bridge half”), but there is no explicit, end-to-end runbook for safely rotating the `NonDefaultMintBurnOfAdapter` token. The docs omit how to briefly stop inbound credits, sweep to zero with the token unpaused, and then switch adapters without races.

Recommendations: Add e.g. the following explicit instructions to the README:

1. Temporarily disable inbound LayerZero routing/messages to the Secondary Chain for the relevant lane.
2. Keep the token unpaused; ensure mint rights on the legacy token and, if `approvalRequired = true`, adapter allowance.
3. Read the adapter’s local chain balance once (`secondaryChainBalance`), mint exactly that amount, and bridge it out via the adapter to zero the balance (optionally submit via a private mempool).
4. Call `setTokenAndApprovalRequired()` with a fresh zero-supply token, then re-enable inbound routing.

Customer’s response: Will be improved before the release.

Fix Review: Acknowledged.

I-04. Use of `ReentrancyGuardTransientUpgradeable` breaks cross-chain compatibility

Description: The `NonDefaultMintBurnOfAdapter` contract imports and uses `ReentrancyGuardTransientUpgradeable`. This specific implementation of a reentrancy guard relies on the `TSTORE` and `TLOAD` opcodes (transient storage), which were introduced in the Dencun hard fork (EIP-1153).

The protocol is intended for deployment on multiple L2s and EVM-compatible chains. However, not all chains have implemented the Dencun upgrade or support these opcodes.

That’s why the other contracts which will be deployed on multiple L2s, such as `CustomToken`, use the regular `ReentrancyGuardUpgradeable`.

Recommendations: Modify the `NonDefaultMintBurnOftAdapter` contract to import and use `ReentrancyGuardUpgradeable` instead of `ReentrancyGuardTransientUpgradeable`. This will align it with the other contracts in the protocol and guarantee its functionality on all EVM-compatible chains.

Customer's response: Addressed in [ac19efd](#).

Fix Review: Confirmed.

I-05. Event hygiene for admin/config setters (missing or insufficient emits)

Description: Several role-gated configuration setters do not emit events, or emit events without old/new values and/or proper indexing, which hinders monitoring and auditability.

- Instances of missing events:
 - `NativeConverter`: `setCustomToken`
 - `NonDefaultMintBurnOftAdapter`: `setTokenAndApprovalRequired`
 - `VaultBridgeTokenPart2`: `setMinimumYieldVaultDeposit`
 - `MigrationManager`: `reinitialize1` (sets `agglayerBridge/_agglayerId`), `reinitialize2` (sets `_wrappedGasToken`)
 - `CustomToken`: `setBridge`, `setNativeConverter` (previously acknowledged)
- Instances of insufficient payload/indexing:
 - `VaultBridgeTokenPart2`: `setYieldVault` (new only; address not indexed), `setYieldRecipient` (new only), `setMinimumReservePercentage` (new only), `setYieldVaultMaximumSlippagePercentage` (new only)
 - `NativeConverter`: `setNonMigratableBackingPercentage` (new only)
 - `WethNativeConverterAgglayer`: `setNonMigratableGasBackingPercentage` (new only)
 - `CustomTokenWethExtension`: `setWethFunctionalityEnabled` (new only)

Recommendations: Emit new events where missing and extend existing events to include old and new values, and index address fields where applicable. Keep naming consistent with "...Set" e.g. `YieldVaultSet(address indexed oldYieldVault, address indexed newYieldVault)`.

Customer's response: Will be considered.

Fix Review: Acknowledged.

I-06. Non-standard ERC-20 `Transfer(address(0), address(0), value)` emitted

Description: When completing migration, the L1 `VaultBridgeTokenPart2` bridges `vbToken` to `address(0)` on the origin network; on L2, `CustomTokenAggLayer.mint()` handles `account == address(0)` by emitting a raw ERC-20 event `Transfer(address(0), address(0), value)` without any actual mint/burn or balance change. This is non-standard for ERC-20 (zero-from denotes mint; zero-to denotes burn; both zero is not a valid balance-changing Transfer) and can mislead or break off-chain indexers and accounting systems that assume standard semantics.

Recommendations: Use a dedicated domain event and stop emitting ERC-20 `Transfer` in the zero-recipient path. If a supply-affecting representation is desired, use a canonical mint-to-self followed by immediate burn to produce standard `Transfer` events.

Customer's response: Addressed in [61163d5](#).

Fix Review: Confirmed.

I-07. Zero-address mint path triggering migration finalization is permitted to `NativeConverter`

Description: In `CustomTokenAggLayer.mint`, the zero-address branch used to finalize migrations is authorized for both `bridge()` and `nativeConverter()`. While current `NativeConverter` flows validate a non-zero receiver before minting, the permission boundary is broader than necessary: `nativeConverter()` is allowed to invoke the zero-address path that is semantically intended for bridge-driven completion.

Recommendations: Check for `(account == address(0) && msg.sender == bridge())`.

Customer's response: Addressed in [a19647a](#).

Fix Review: Confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.